

תרגול 5 - Javascript intro

כל הזכויות שמורות לקרן יעקב 2025

על המצגת

- במצגת זו יש גם חומר תיאורטי וגם דוגמאות קוד. המצגת מכילה גם חומר שלא למדנו בהרצאה וזו אולי הפעם הראשונה שתפגשו אותו.
- הרצת JS בתרגול - לשם נוחות הלמידה, בחלק מהדוגמאות במצגת זו הקוד JS כתוב בתחתית ה-HTML, סוף ה-body (ולא בדף js נפרד כמו שאתם תתכנתו).
- בסוף כל נושא מופיע הקוד להעתקה

נושאי התרגול

1. הדפסות לקונסול
2. [debugger](#) keyword
3. Debugging
4. Variables
5. Scopes
6. Hoisting
7. [this](#) keyword
8. events

1- הדפסות ל-Console

- נלמד לעבוד נכון עם הקונסול ולדבג עם ההודעה המתאימה לכל מאורע.
- הודעת alert לצורך דיבאג אינה מקובלת!
- נבין ונזהה את ההבדלים בין דרגות הקונסול המקובלות:

- Log (common)
- Error (common)
- Warn
- Debug

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Javascript - console printing</title>
5   </head>
6
7   <body>
8     <script>
9       console.log("log");
10      console.error("error");
11      console.warn("warn");
12      console.debug("debug");
13    </script>
14  </body>
15 </html>
```



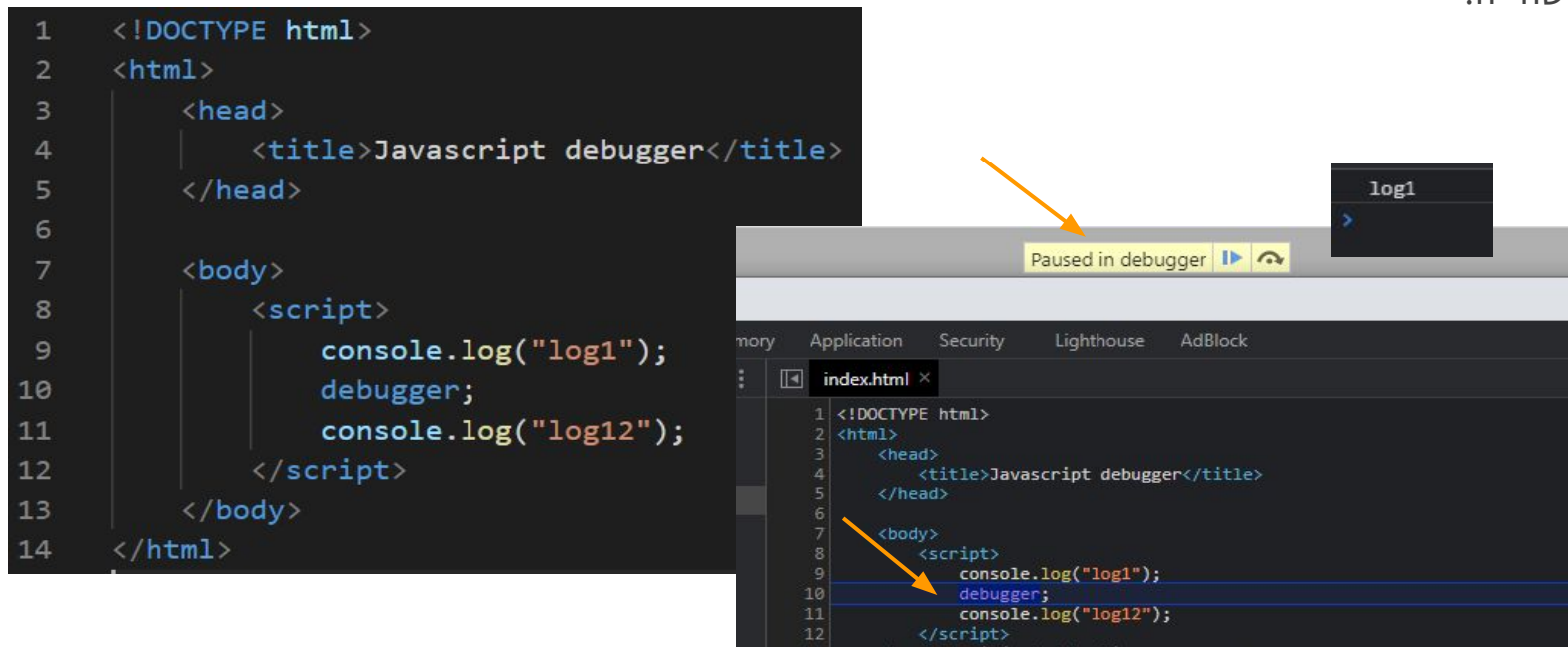
1 - הדפסות ל-Console

```
<!DOCTYPE html>
<html>
  <head>
    <title>Javascript - console printing</title>
  </head>

  <body>
    <script>
      console.log("log");
      console.error("error");
      console.warn("warn");
      console.debug("debug");
    </script>
  </body>
</html>
```

2- 'Debugger' keyword

- אנו יכולים להוסיף לתוכנית שלנו את המילה השמורה **debugger**. כשהקוד יגיע לשורה זו - הריצה של הדפדפן תהיה בהשהייה.



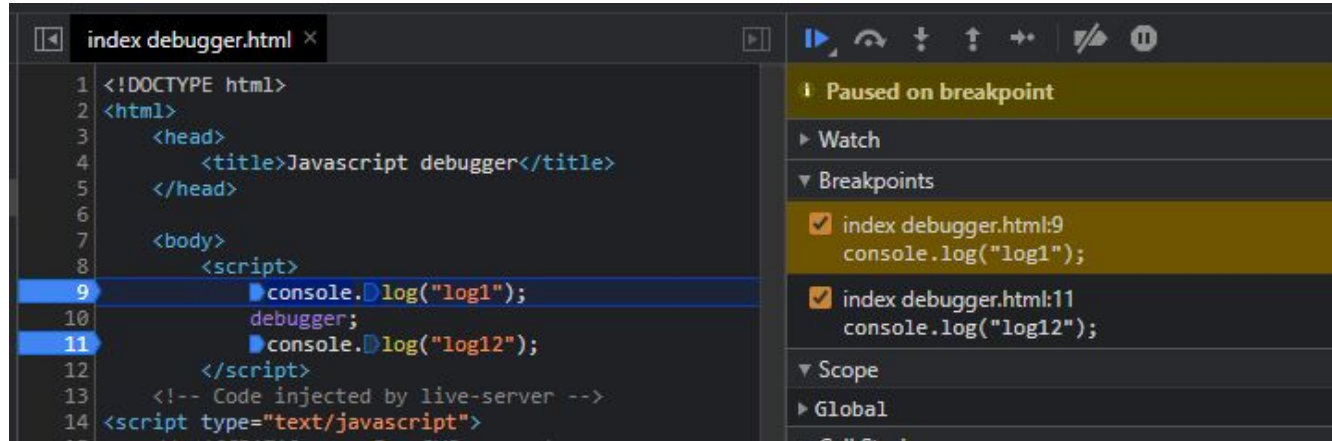
2- 'Debugger' keyword

```
<!DOCTYPE html>
<html>
  <head>
    <title>Javascript debugger</title>
  </head>

  <body>
    <script>
      console.log("log1");
      debugger;
      console.log("log12");
    </script>
  </body>
</html>
```

3- Debugging

- מומלץ לכתוב את הקוד באופן שהוא מספר סיפור מלמעלה למטה. משמע, למעלה פונקציה כללית, למטה הפונקציה הספציפית או אירועים שמתחילים את שרשרת הפונקציות. בכל מקרה, כל פונקציה מוגדרת לפני שקוראים לה.
- בחלון הקוד לוחצים על שורה שמאפשרת להוסיף בה breakpoint
- מתחילים להריץ את הקוד, האפשרויות: לבצע ולדלג, להיכנס פנימה וכן הלאה.
- הפרמטרים, אירועים ואובייקטים שבשימוש מופיעים בצד ימין



4- Variables

1. let and const keywords :

The keyword "let" enables the users to define variables and on the other hand, "const" enables the users to define constants. Variables were previously declared using "var" which had function scope and were hoisted to the top. It means that a variable can be used before declaration. But, the "let" variables and constants have block scope which is surrounded by curly-braces "{}" and cannot be used before declaration.

```
let i = 10;  
console.log(i);    //Output 10
```

```
const PI = 3.14;  
console.log(PI);   //Output 3.14
```

4- Variables

```
const a = 1;  
a = 2;  
console.log(a);
```

```
let a = 1;  
a = 2;  
console.log(a);
```

מה הפלט בכל אחד מהמקרים?



4- Variables

```
const a = 1;  
a = 2;  
console.log(a);
```

✖ ▶ Uncaught TypeError: Assignment to constant variable.
at [index.html:9:15](#)

```
let a = 1;  
a = 2;  
console.log(a);
```

2

מה הפלט בכל אחד מהמקרים?



5- JS Scopes

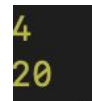
- בלוק הוא אזור תחום בסוגריים מסולסלים, JS מאפשר להגדיר משתנה ל- block של קוד
- בדוגמה הבאה, מוגדר b רק לבלוק של ה- if

הפלט הוא:

```
var a = 10; //a is global  
let b = 20;
```

```
if (a) {  
  var a = 4; //a is still global  
  let b = 6; //let is blocked  
}
```

```
console.log(a);  
console.log(b);
```



4
20

5- JS Scopes

התרגיל הבא לקוח מתוך מאגר שאלות סטנדרטיות בראיונות עבודה. נתון הקוד הבא:

```
let a = 8;
let b = 2;

(function(){
  a = 10;
  b = 5;
  console.log("a + b = ", a + b);
})();

console.log("a = ", a);
console.log("b = ", b);
```

- מה יהיה הפלט? למה?
- איך נשנה את התוצאה?

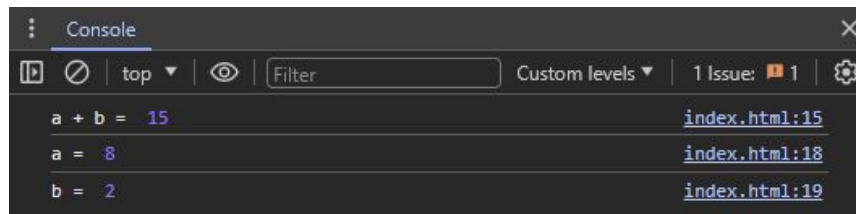
5- JS Scopes

התרגיל הבא לקוח מתוך מאגר שאלות סטנדרטיות בראיונות עבודה. נתון הקוד הבא:

```
let a = 8;
let b = 2;

(function(){
  a = 10;
  b = 5;
  console.log("a + b = ", a + b);
})();

console.log("a = ", a);
console.log("b = ", b);
```



מה יהיה הפלט? למה?

איך נשנה את התוצאה?

5- JS Scopes

```
<!DOCTYPE html>
<html>
  <head>
    <title>1B - Javascript scopes</title>
  </head>

  <body>
    <script>
      let a = 8;
      let b = 2;

      (function() {
        a = 10;
        b = 5;
        console.log('a + b = ', a + b);
      })();

      console.log('a = ', a);
      console.log('b = ', b);
    </script>
  </body>
</html>
```

5- JS Scopes A

● נתון הקוד הבא, מה יודפס לקונסול? למה?

```
8 <script>
9   let globalVar = 1;
10
11   function functionName() {
12     let functionVar = 2;
13     console.log('inside function', functionVar);
14   }
15
16   {
17     let blockVar = 3;
18     console.log(blockVar);
19   }
20
21   functionName();
22   console.log(globalVar);
23 </script>
```

5- JS Scopes A

● נתון הקוד הבא, מה יודפס לקונסול? למה?

```
8 <script>
9   let globalVar = 1;
10
11   function functionName() {
12     let functionVar = 2;
13     console.log('inside function', functionVar);
14   }
15
16   {
17     let blockVar = 3;
18     console.log(blockVar);
19   }
20
21   functionName();
22   console.log(globalVar);
23 </script>
```

```
3
inside function 2
1
```

5- JS Scopes A

```
<!DOCTYPE html>
<html>
  <head>
    <title>Javascript variables</title>
  </head>
  <body>
    <script>
      let globalVar = 1;

      function functionName() {
        let functionVar = 2;
        console.log('inside function', functionVar);
      }

      {
        let blockVar = 3;
        console.log(blockVar);
      }

      functionName();
      console.log(globalVar);
    </script>
  </body>
</html>
```

6- Hoisting

- לפי ההתנהגות הדיפולטית של Javascript – העברת כל ההצהרות לתחילת הסקופ, גם אם הן פיזית לא נמצאות שם. רק הצהרות נקראות קודם, לא ההשמות (גם אם הן ביחד)
- JS בעצמו מניף (Hoisting) את כל ההצהרות על המשתנים ועל פונקציות (בלי לקרוא להן) לראש הדף (או לראש ה-scope) כדי לשמור מקום בזיכרון, כאשר החוק אינו תקף להשמות.
- משתנים המוגדרים ע"י let, const גם נקראים קודם אבל לא מאותחלים, המשמעות היא שהקוד מודע למשתנה, אבל אם ננסה להשתמש בו נקבל שגיאת Reference Error.
- המשתנה var לעומת זאת (שלא נשתמש בו בקורס) יקבל את הערך undefined
- יתר הקוד נקרא לפי סדר הופעתו במסמך

```
console.log (greeter);  
var greeter = "say hello"
```

it is interpreted as this:

```
var greeter;  
console.log(greeter); // greeter is undefined  
greeter = "say hello"
```

6- Hoisting

התרגיל הבא לקוח מתוך מאגר שאלות סטנדרטיות בראיונות עבודה

נתון הקוד הבא-

```
index.html X
Tar5 - JS > 3-Hoisting > index.html > ...
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>3 - Javascript hoisting</title>
5     <script src="js/scripts.js"> </script>
6   </head>
7
8   <body>
9     <script>
10      setAllVals();
11    </script>
12  </body>
13 </html>
```

```
JS scripts.js X
Tar5 - JS > 3-Hoisting > js > JS scripts.js > ...
1 function setAllVals() {
2   console.log(x);
3   console.log(getIt());
4
5   var x = 1;
6
7   function getIt() {
8     return 2;
9   }
10 }
```

מה ה- output? למה?

מה יקרה אם נחליף את ההצהרה מ- var ב- let?

6- Hoisting

התרגיל הבא לקוח מתוך מאגר שאלות סטנדרטיות בראיונות עבודה

נתון הקוד הבא-

```
index.html X
Tar5 - JS > 3-Hoisting > index.html > ...
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>3 - Javascript hoisting</title>
5     <script src="js/scripts.js"> </script>
6   </head>
7
8   <body>
9     <script>
10      setAllVals();
11    </script>
12  </body>
13 </html>
```

```
JS scripts.js X
Tar5 - JS > 3-Hoisting > js > JS scripts.js > ...
1 function setAllVals() {
2   console.log(x);
3   console.log(getIt());
4
5   var x = 1;
6
7   function getIt() {
8     return 2;
9   }
10 }
```

```
Console X
[ ] [ ] top [ ] Filter Custom levels 1 Issue: 1 [ ]
undefined scripts.js:2
2          scripts.js:3
```

מה ה-output? למה?

מה יקרה אם נחליף את ההצהרה מ- var ב- let? שגיאה כי x לא אותחל

6- Hoisting

```
<!DOCTYPE html>
<html>
  <head>
    <title>3 - Javascript
    hoisting</title>
    <script src="js/scripts.js">
</script>
  </head>

  <body>
    <script>
      setAllVals();
    </script>
  </body>
</html>
```

```
function setAllVals() {
  console.log(x);
  console.log(getIt());

  var x = 1;

  function getIt() {
    return 2;
  }
}
```

7- 'this' keyword

- השימוש במילה השמורה 'this' הוא תלוי בהקשר
- התרגיל הבא לקוח מתוך מאגר שאלות סטנדרטיות בראיונות עבודה:
מה ה-output? למה?
- מה יקרה אם נשתמש בפונקציית חץ בשורה 7?

```
1  var fullName = 'John Doe';
2
3  var obj = {
4    fullName: 'Colin Ihrig',
5    prop: {
6      fullName: 'Aurelio De Rosa',
7      getFullName: function() {
8        return this.fullName;
9      }
10   }
11 };
12
13 console.log(obj.prop.getFullName());
14
15 var test = obj.prop.getFullName;
16
17 console.log(test());
```

7- 'this' keyword

- השימוש במילה השמורה 'this' הוא תלוי בהקשר
- התרגיל הבא לקוח מתוך מאגר שאלות סטנדרטיות בראיונות עבודה:

מה ה-output? למה?

Aurelio De Rosa

John Doe

- מה יקרה אם נשתמש בפונקציית חץ בשורה 7?

```
getFullname: () => {  
  return this.fullname;  
}
```

John Doe

John Doe

>

```
1  var fullName = 'John Doe';  
2  
3  var obj = {  
4    fullName: 'Colin Ihrig',  
5    prop: {  
6      fullName: 'Aurelio De Rosa',  
7      getFullname: function() {  
8        return this.fullName;  
9      }  
10   }  
11 };  
12  
13 console.log(obj.prop.getFullname());  
14  
15 var test = obj.prop.getFullname;  
16  
17 console.log(test());
```

7- 'this' keyword

```
var fullName = 'John Doe';
```

```
var obj = {  
  fullName: 'Colin Ihrig',  
  prop: {  
    fullName: 'Aurelio De Rosa',  
    getFullName: function() {  
      return this.fullName;  
    }  
  }  
};
```

```
console.log(obj.prop.getFullName());
```

```
var test = obj.prop.getFullName;
```

```
console.log(test());
```

8- events

- הפונקציה המובנית `getElementById` פונה ישירות לאובייקט שה-ID שלו מוכל בסוגריים

- למדנו שה-id הוא ייחודי לאובייקט אחד בלבד בעמוד HTML

- בדוגמה הבאה אנו פונים לאלמנט שיש לו id בשם btn: `document.getElementById("btn")`

- בתרגיל הבא, נרצה לשנות את הטקסט בכותרת בלחיצה על כפתור

- השלבים:

- ב-HTML נגדיר כפתור

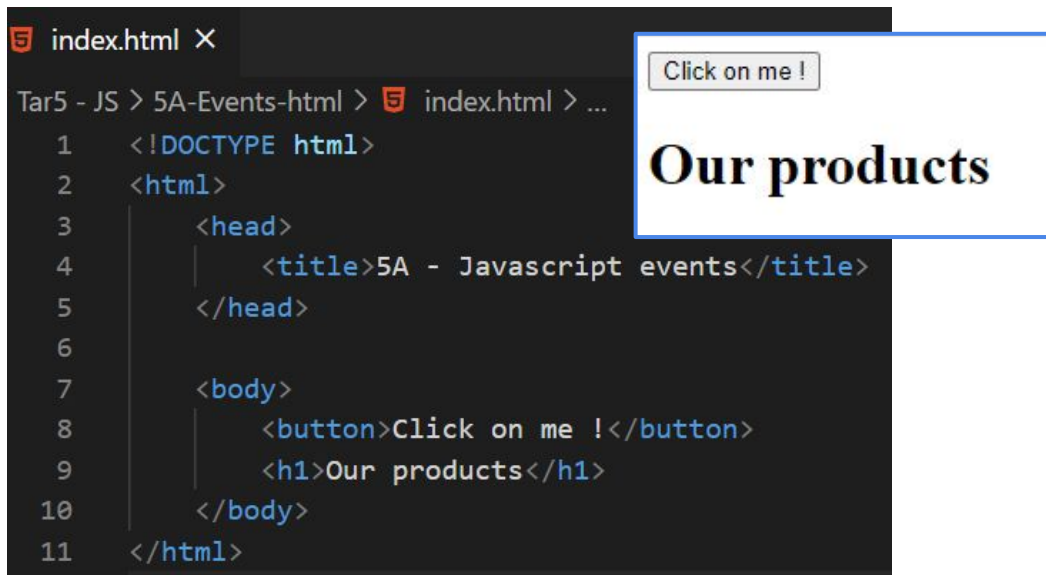
- ב-HTML נגדיר כותרת ראשית עם טקסט

- ב-JS נגדיר שלחיצה על כפתור תשנה את הטקסט שבכותרת (רק בתרגול, נוכל לשים בתחתית ה-HTML)

- כל שלב נדבג ע"י הדפסות לקונסול (זמניות), במידת האפשר

8- events A

● בשלב הראשון ניצור את האובייקטים ב-HTML ונריץ כדי לראות שאנו עומדים בדרישות



```
index.html X
Tar5 - JS > 5A-Events-html > index.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>5A - Javascript events</title>
5    </head>
6
7    <body>
8      <button>Click on me !</button>
9      <h1>Our products</h1>
10   </body>
11 </html>
```

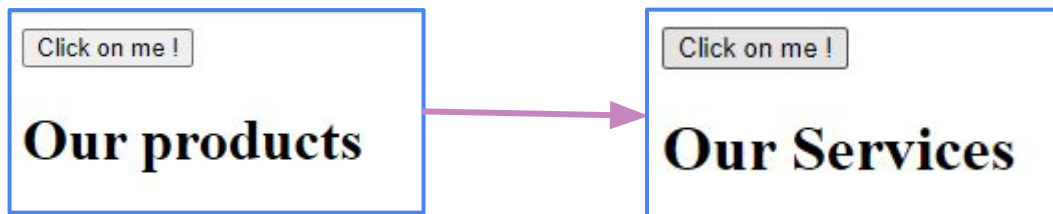
8- events A

```
<!DOCTYPE html>
<html>
  <head>
    <title>2A - Javascript events</title>
  </head>
  <body>
    <button>Click on me !</button>
    <h1>Our products</h1>
  </body>
</html>
```

8- events B

● קיימים האובייקטים ב-HTML. השלב הבא הוא להוסיף את ההתנהגות ע"י קוד Javascript

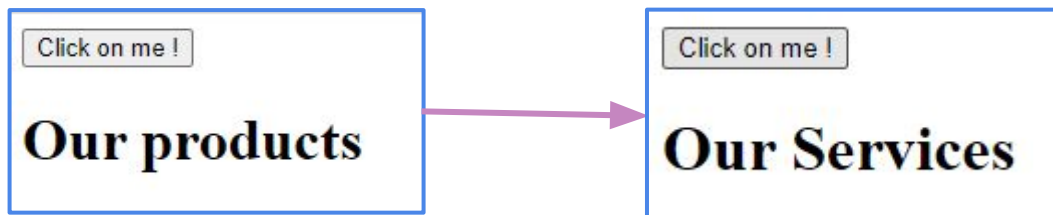
- נמצא מי הטריגר - מי מתחיל את הפעולה?
- לאחר ביצוע הטריגר - מה הפעולה שצריך לבצע?



8- events B

● קיימים האובייקטים ב-HTML. השלב הבא הוא להוסיף את ההתנהגות ע"י קוד Javascript

- נמצא מי הטריגר - מי מתחיל את הפעולה?
- לאחר ביצוע הטריגר - מה הפעולה שצריך לבצע?



- נשתמש בפונקציה המוכנה onclick כדי לתפוס את הטריגר
- נשתמש ב- innerHTML כדי להחליף את הכותרת

8- events B

- עכשיו נקרא את הקוד ונבין מה כל חלק עושה
- שימו לב בשורה 17 לסוגריים הריקים - למה הם משמשים? במה אפשר להחליף אותם?
- אם אנחנו רוצים להחליף את הטקסט בחזרה לטקסט הראשון מה אפשר לעשות? ובקוד?

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>2B - Javascript events</title>
5    </head>
6    <body>
7      <button id="btn">Click on me !</button>
8      <h1 id="pageTitle">Our products</h1>
9
10     <script>
11       document.getElementById("btn").onclick = () => {
12         document.getElementById("pageTitle").innerHTML = "Our Services";
13       };
14     </script>
15   </body>
16 </html>
```

8- events B

```
<!DOCTYPE html>
<html>
  <head>
    <title>2B - Javascript events</title>
  </head>
  <body>
    <button id="btn">Click on me !</button>
    <h1 id="pageTitle">Our products</h1>

    <script>
      document.getElementById("btn").onclick = () => {
        document.getElementById("pageTitle").innerHTML = "Our Services";
      };
    </script>
  </body>
</html>
```